

Preparing for the F.Y.B.Sc.(IT) Semester II university practicals can be demanding, especially with the distinct curriculum focus in the IT track compared to CS. Advanced topics like multithreading, GUI development with Tkinter, and object-oriented concepts require precise execution.

Here are the complete, executable Python solutions for all 15 practical slips for the IT152MJP Lab Course. They are kept clean and concise so they are easy to practice and memorize.

Slip 1

****Q.1) Write a program that reads all the lines from a file named "data.txt" and prints each line of the console.****

```
```python
Create a dummy file for testing purposes first
with open("data.txt", "w") as f:
 f.write("Line 1\nLine 2\nLine 3")
```

**# Solution**

```
with open("data.txt", "r") as file:
 for line in file:
 print(line.strip())
```

...

**\*\*Q.2) Write a Python script that creates two threads. one thread should print numbers from 1 to 5 with a delay of 1 sec between each print, and the other thread should print letter from A to E with the same delay\*\***

```
```python
import threading
import time
```

```
def print_numbers():
    for i in range(1, 6):
        print(i)
        time.sleep(1)
```

```
def print_letters():
    for letter in ['A', 'B', 'C', 'D', 'E']:
        print(letter)
        time.sleep(1)
```

```
t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)
```

```
t1.start()
t2.start()
```

```
t1.join()
t2.join()
```

...

Slip 2

****Q.1)** Write a regular expression to validate a phone number format(e.g...(123) 456-789).******

```
```python
```

```
import re
```

```
def validate_phone(number):
 pattern = r"^\(\d{3}\) \d{3}-\d{3}$"
 if re.match(pattern, number):
 print(f"{number} is Valid")
 else:
 print(f"{number} is Invalid")
```

```
validate_phone("(123) 456-789")
```

```
validate_phone("123-456-7890")
```

```
...
```

**\*\*Q.2)** Write a Python program demonstrating Hierarchical Inheritance by creating a base class Shape with a method area(). Derive two classes Circle and Rectangle from Shape. Implement the area() method in both derived classes. **\*\***

```
```python
```

```
import math
```

```
class Shape:
```

```
    def area(self):
        pass
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return math.pi * (self.radius ** 2)
```

```
class Rectangle(Shape):
```

```
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
```

```
c = Circle(5)
```

```
r = Rectangle(4, 6)
```

```
print(f"Circle Area: {c.area():.2f}")
```

```
print(f"Rectangle Area: {r.area()}")
```

```
...
```

Slip 3

****Q.1)** Write a Python script that opens a file named "example.txt" in write mode, writes the string "Hello, World!" to it, and then closes the file. ******

```
```python
file = open("example.txt", "w")
file.write("Hello, World!")
file.close()

print("Data written successfully.")
```

...

**\*\*Q.2)** Write a Python program that demonstrates multiple inheritance by creating two base classes Person and Employee, and a derived class Manager that inherits from both Person and Employee. **\*\***

```
```python
class Person:
    def __init__(self, name):
        self.name = name

class Employee:
    def __init__(self, emp_id):
        self.emp_id = emp_id

class Manager(Person, Employee):
    def __init__(self, name, emp_id, department):
        Person.__init__(self, name)
        Employee.__init__(self, emp_id)
        self.department = department

    def display(self):
        print(f"Name: {self.name}, ID: {self.emp_id}, Dept: {self.department}")

m = Manager("Alice", 101, "IT")
m.display()

```
```

#### ### Slip 4

**\*\*Q.1)** Write a program that reads all the lines from a file named "data.txt" and prints each line to the console.\*\*

\*(Note: Duplicate of Slip 1, Q1)\*

```
```python
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

```

**\*\*Q.2)** Write a Python GUI program to import Tkinter package and create window and set its title.\*\*

```
```python
import tkinter as tk

root = tk.Tk()
root.title("My First Tkinter Window")
root.geometry("300x200")

root.mainloop()
```

```

#### ### Slip 5

**\*\*Q.1)** Write a script that checks if a file named "config.ini" exists in the current directory. If it exists, print "File exists", otherwise print "File does not exist".\*\*

```
```python
import os

if os.path.exists("config.ini"):
    print("File exists")
else:
    print("File does not exist")
```

```

**\*\*Q.2)** Write a Python GUI program to import Tkinter package and create a window. Set its title and add a label to the window.\*\*

```
```python
import tkinter as tk

root = tk.Tk()
root.title("Tkinter Label Window")

label = tk.Label(root, text="Welcome to Advanced Python!")
label.pack(pady=20)

root.mainloop()
```

```

### Slip 6

**\*\*Q.1)** Write a Python script that reads a binary file named "image.png" and writes its content to another file named "copy\_image.png".\*\*

```
```python
try:
    with open("image.png", "rb") as file_in:
        data = file_in.read()

    with open("copy_image.png", "wb") as file_out:
        file_out.write(data)
    print("Image copied successfully.")
except FileNotFoundError:
    print("Please ensure 'image.png' exists in the directory first.")
```
```

**\*\*Q.2)** Write a Python program using Tkinter to create a simple window with the title "Hello World" and a size of 300x200 pixels.\*\*

```
```python
import tkinter as tk

root = tk.Tk()
root.title("Hello World")
root.geometry("300x200")

root.mainloop()
```
```

### Slip 7

**\*\*Q.1) Write a Python program to demonstrate use of regular expression with suitable application.\*\***

```
```python
import re

# Application: Email Validator
def is_valid_email(email):
    pattern = r"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$"
    if re.match(pattern, email):
        return True
    return False

test_email = "student@sppu.edu"
print(f'Is '{test_email}' valid? {is_valid_email(test_email)}')
```
```

**\*\*Q.2) Create a Tkinter application that includes the following widgets: a label, a button, an entry field, and a text box. Add appropriate text to each widget.\*\***

```
```python
import tkinter as tk

root = tk.Tk()
root.title("Widget Demo")

tk.Label(root, text="Enter your name:").pack()
tk.Entry(root).pack()
tk.Button(root, text="Submit").pack()

text_box = tk.Text(root, height=5, width=30)
text_box.insert(tk.END, "This is a multi-line text box.")
text_box.pack()

root.mainloop()
```
```

### Slip 8

**\*\*Q.1) Write a Program to demonstrate concept of threading and multitasking in Python.\*\***

```
```python
import threading
import time

def task(name, delay):
    print(f"Task {name} starting.")
    time.sleep(delay)
    print(f"Task {name} completed.")

t1 = threading.Thread(target=task, args=("A", 2))
t2 = threading.Thread(target=task, args=("B", 1))

t1.start()
t2.start()

t1.join()
t2.join()
print("All tasks finished.")
```

```

**\*\*Q.2) Write a Python program that creates a Tkinter window with a label displaying the text "Hello, Tkinter!" in red color and with a font size of 20.\*\***

```
```python
import tkinter as tk

root = tk.Tk()
root.geometry("300x200")

label = tk.Label(root, text="Hello, Tkinter!", fg="red", font=("Arial", 20))
label.pack(pady=50)

root.mainloop()
```

```

### Slip 9

**\*\*Q.1) Write a Python program using a regular expression to find all occurrences of the sequence "abc" in a given string.\*\***

```
```python
```

```
import re
```

```
text = "abc is the start of the alphabet. xxyyabc is another test, and ABC is uppercase."
```

```
matches = re.findall(r"abc", text)
```

```
print(f"Found {len(matches)} occurrences:", matches)
```

```
```
```

**\*\*Q.2) Create a Tkinter application with a button that changes the text of a label when clicked. Implement the event handling for the button click.\*\***

```
```python
```

```
import tkinter as tk
```

```
def change_text():
```

```
    label.config(text="Text has been changed!")
```

```
root = tk.Tk()
```

```
root.geometry("300x200")
```

```
label = tk.Label(root, text="Original Text")
```

```
label.pack(pady=20)
```

```
btn = tk.Button(root, text="Click Me", command=change_text)
```

```
btn.pack()
```

```
root.mainloop()
```

```
```
```



### Slip 10

**\*\*Q.1)** Write a Python program that uses regular expressions to find sequences of digits in a given string where the sequence length is exactly 3.

```
python
import re

text = "My numbers are 12, 123, 4567, and 890."
matches = re.findall(r"\b\d{3}\b", text)

print("Exact 3-digit sequences:", matches)
```

...

**\*\*Q.2)** Write a program where a Department class has a list of Employee objects. Add methods to display all employees in a department.

```
python
class Employee:
 def __init__(self, name):
 self.name = name

class Department:
 def __init__(self, dept_name):
 self.dept_name = dept_name
 self.employees = []

 def add_employee(self, emp):
 self.employees.append(emp)

 def display_employees(self):
 print(f"Employees in {self.dept_name}:")
 for emp in self.employees:
 print("- " + emp.name)

it_dept = Department("IT")
it_dept.add_employee(Employee("Rahul"))
it_dept.add_employee(Employee("Priya"))

it_dept.display_employees()
```

...

### Slip 11

**\*\*Q.1)** Write a Python program that matches a string that has an a followed by three 'b'.\*\*

```
```python
```

```
import re
```

```
def match_string(text):
```

```
    if re.search(r"ab{3}", text):
```

```
        print(f'{text}' contains the match.)
```

```
    else:
```

```
        print(f'{text}' does not contain the match.)
```

```
match_string("abbb")
```

```
match_string("abb")
```

```
match_string("xabbb")
```

```
```
```

**\*\*Q.2)** Create a class Employee with instance variables name and salary, and a class variable company\_name. Add methods to display the details of an employee and to change the company name for all employees.\*\*

```
```python
```

```
class Employee:
```

```
    company_name = "Tech Solutions"
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

```
        self.salary = salary
```

```
    def display(self):
```

```
        print(f'Name: {self.name}, Salary: {self.salary}, Company:  
{Employee.company_name}')
```

```
    @classmethod
```

```
    def change_company(cls, new_name):
```

```
        cls.company_name = new_name
```

```
e1 = Employee("Amit", 50000)
```

```
e1.display()
```

```
Employee.change_company("Global Tech")
```

```
e1.display()
```

```
```
```

#### Slip 12

**\*\*Q.1)** Write a program that reads all the lines from a file named "data.txt" and prints each line to the console.\*\*

\*(Note: Duplicate of Slip 1, Q1)\*

```
```python
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

```

**\*\*Q.2)** Write a class Car with instance variables make and model, and a class variable wheels. Demonstrate the use of the instance and class namespaces by creating two car objects and modifying their attributes.\*\*

```
```python
class Car:
    wheels = 4 # Class variable

    def __init__(self, make, model):
        self.make = make    # Instance variable
        self.model = model  # Instance variable
```

```
car1 = Car("Toyota", "Corolla")
car2 = Car("Honda", "Civic")
```

```
# Modifying instance attribute
car1.model = "Camry"
```

```
# Modifying class attribute for all instances
Car.wheels = 5
```

```
print(f'{car1.make} {car1.model} has {car1.wheels} wheels.')
print(f'{car2.make} {car2.model} has {car2.wheels} wheels.')
```

```

### Slip 13

**\*\*Q.1)** Create a program that reads a binary file named "image.png" and writes its content to another file named "copy\_image.png".\*\*

\*(Note: Duplicate of Slip 6, Q1)\*

```
```python
with open("image.png", "rb") as f_in:
    with open("copy_image.png", "wb") as f_out:
        f_out.write(f_in.read())
```

```

**\*\*Q.2)** Write a Python class Person with attributes name and age. Include a method display that prints the name and age of the person. Demonstrate creating an instance of Person and calling the display method.\*\*

```
```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Person Name: {self.name}, Age: {self.age}")
```

```
p = Person("Karan", 20)
p.display()
```

```

### Slip 14

**\*\*Q.1)** Write a Python program using a regular expression to find all occurrences of the sequence "abc" in a given string.

**\*(Note: Duplicate of Slip 9, Q1)\***

```
```python
import re
text = "abc 123 abc xyz"
print(re.findall(r"abc", text))
```

```

**\*\*Q.2)** Write two classes, Student and Course. The Student class should have attributes name and age, and a method display. The Course class should have attributes course\_name and students (a list of Student objects). Add a method to Course that adds a student to the course and displays all students in the course.

```
```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        return f"{self.name} (Age: {self.age})"

class Course:
    def __init__(self, course_name):
        self.course_name = course_name
        self.students = []

    def add_student(self, student):
        self.students.append(student)

    def display_students(self):
        print(f"Students in {self.course_name}:")
        for student in self.students:
            print("- " + student.display())

course = Course("Advanced Python")
course.add_student(Student("Neha", 19))
course.add_student(Student("Rohit", 20))

course.display_students()
```

```

### Slip 15

**\*\*Q.1)** Write a Python program that matches a string that has an a followed by zero or more b's.

```
```python
import re

def match_zero_or_more_b(text):
    if re.fullmatch(r"ab*", text):
        print(f'{text} is a Match')
    else:
        print(f'{text} is Not a Match')
```

```
match_zero_or_more_b("a")
match_zero_or_more_b("ab")
match_zero_or_more_b("abbb")
match_zero_or_more_b("ac")
```

...

****Q.2)** Create a base class Vehicle with an attribute speed and a method drive(). Derive a class Car from Vehicle and add an attribute fuel_type. Demonstrate creating an instance of Car

```
```python
class Vehicle:
 def __init__(self, speed):
 self.speed = speed

 def drive(self):
 print(f'Driving at {self.speed} km/h')
```

```
class Car(Vehicle):
 def __init__(self, speed, fuel_type):
 super().__init__(speed)
 self.fuel_type = fuel_type
```

```
my_car = Car(100, "Petrol")
my_car.drive()
print(f'Fuel type: {my_car.fuel_type}')
```

...